ACR-VLA: Adaptive Conformal Refinement for Frozen Flow-Matching VLAs

Anonymous Author(s)
Affiliation
Address
email

Abstract: We propose ACR-VLA, a lightweight inference-time correction pipeline for frozen flow-matching Vision-Language-Action models. Drawing only $K{=}2$ stochastic action samples in a single batched forward pass, ACR-VLA uses their pairwise L_2 distance as a per-step uncertainty signal, well-calibrated against ten-sample variance ($\rho{=}0.9979$) at zero additional latency, gating a 327K-parameter Correction Residual Module (CRM). On SmolVLA (0.4B) across four LIBERO suites, CRM applied unconditionally degrades two suites (-1.2 pp Object, -2.2 pp Long), while ACR-VLA's adaptive gate exceeds unconditional correction on all four suites by selectively correcting only high-uncertainty steps. The gate's trigger rate correlates monotonically with suite difficulty, allocating inference-time compute where the VLA is least confident. A cross-VLA analysis on $\pi_{0.5}$ (3.6B, 94–99% baseline) shows that unconditional correction degrades near-expert VLAs by up to 17 pp, but adaptive gating recovers 19–38% of the loss on 3/4 suites, confirming the gate's value as a damage-limiting mechanism across capability levels.

Keywords: Vision-Language-Action Models, Inference-Time Refinement, Robotic Manipulation

1 Introduction

Deploying a pretrained Vision-Language-Action (VLA) model on a new task usually requires fine-tuning on task demonstrations, yet collecting that data is expensive and the resulting checkpoint may not transfer further. A more attractive route keeps the VLA frozen and improves action quality at inference time. Flow-matching VLAs [1, 2, 3] are well suited to this setting because each forward pass produces a stochastic action sample, enabling multi-sample selection and correction without retraining [4]. The naive approach draws $K{=}10$ samples and applies unconditional correction at every step, but most control steps are easy, and we show that correcting them adds noise without benefit.

Recent work touches parts of this question without answering it. A2C2 [5] learns a correction head on action chunks but applies it unconditionally and addresses temporal staleness rather than initial action quality. ReconVLA [6] quantifies uncertainty over frozen VLAs but does not learn a corrective residual. SC-VLA [7] uses reinforcement learning and therefore does not stay weight-frozen. VLA-Pilot [8] performs evolutionary search at 2.41 s per control step, ruling out closed-loop deployment. None address the core question: can a frozen VLA decide *per step* whether it needs correction at all?

We propose **ACR-VLA**, an adaptive inference-time pipeline for frozen flow-matching VLAs. ACR-VLA draws only $K{=}2$ samples in a single batched forward pass and uses their pairwise L_2 distance as a per-step uncertainty probe. This probe is well-calibrated: Spearman $\rho{=}0.9979$ against $K{=}10$ sample variance, whereas three alternatives all yield $\rho < 0.16$. When the probe falls below a threshold τ, the pipeline outputs the argmin-to-centroid sample directly; when it exceeds τ, a

Submitted to the 10th Conference on Robot Learning (CoRL 2026). Do not distribute.

lightweight Correction Residual Module (CRM, ∼327K parameters, $<1\%$ of the backbone) applies a bounded residual.

On SmolVLA [2] (0.4B) across four LIBERO suites [9], unconditional CRM correction (ε=0.05) degrades performance on 2 of 4 suites (-1.2 pp Object, -2.2 pp Long). ACR-VLA's adaptive gate triggers correction only on high-uncertainty steps (4–20% of steps), achieving directionally positive results on high-uncertainty suites while preserving baseline on confident ones. Across all four suites, ACR-VLA exceeds CRM always-on (Δ_{CRM}=+1.0 to +1.9 pp), using $5\times$ fewer VLA forward passes. A cross-VLA analysis on $\pi_{0.5}$ [10] (3.6B, 94–99% baseline) shows that unconditional correction degrades near-expert VLAs by up to -17 pp, but adaptive gating recovers 19–38% of the loss on 3/4 suites, confirming the gate as a safety mechanism even when correction itself provides limited value. A boundary analysis on OpenVLA-OFT [11] (7B, deterministic) confirms that stochastic sample diversity is a prerequisite.

The key finding is mechanistic: *when* to correct matters more than correction strength. CRM always-on applies the same learned residual to every step, including the ∼80–90% where the VLA is already confident; corrections on these steps are noise. ACR-VLA's adaptive gate triggers CRM only on high-uncertainty steps (trigger rate 4–20%, varying with suite difficulty), preserving the VLA's own output elsewhere. Threshold sweeps on two suites confirm an inverted-U response (single-seed; Appendix B): too much correction is as harmful as too little, establishing gating as a necessary safety mechanism.

Our contributions are:

1. **ACR-VLA pipeline** (Section 3): an adaptive K=2 uncertainty-gated correction pipeline for frozen flow-matching VLAs. A lightweight CRM fires only when per-step uncertainty exceeds a calibrated threshold, using $5\times$ fewer VLA forward passes than K=10 methods while matching or exceeding their task performance.
2. **Gating as safety mechanism** (Section 4.2): CRM without gating degrades 2 of 4 suites; adaptive gating prevents this degradation while maintaining baseline on confident suites. ACR-VLA exceeds unconditional correction on all four suites.
3. **Gating necessity analysis** (Section 4.3): correction without gating is unreliable across suites; selection without correction is insufficient. The adaptive gate is the essential component.
4. **Applicability boundaries** (Section 4.5): cross-VLA analysis on $\pi_{0.5}$ (3.6B, 94–99% baseline) reveals that unconditional correction degrades near-expert VLAs by up to -17 pp, but adaptive gating recovers 19–38% of the loss on 3/4 suites. Deterministic VLAs (OpenVLA-OFT) produce near-identical samples that make multi-sample methods vacuous.

2 Related Work

VLA robustness. Vision-language-action models have progressed from tokenized prediction [12, 11, 13] to generative approaches based on diffusion [14] and flow matching [1, 2]. Robustness evaluations [15] show that current VLAs lose over 65 pp under deployment perturbations. Training-time defenses (SRPO [16], RobustVLA [17], DiG-Flow [18], TT-VLA [19]) improve robustness but require retraining or reward engineering.

Inference-time correction. Among frozen-VLA methods, ReconVLA [6] provides uncertainty quantification but no learned correction; SC-VLA [7] requires task-level RL; VLA-Pilot [8] uses evolutionary optimization at 2.41 s per step. A2C2 [5] is the closest concurrent work: it learns a correction head addressing *temporal staleness* within action chunks. Our pipeline targets a complementary problem, namely *initial action quality*, with multi-sample centering, explicit magnitude control, and adaptive per-step compute allocation. Residual policy learning [20, 21] shares the additive-correction structure; magnitude control is the central challenge, addressed by gating [22], RL-bounded clipping [23], or decay [24]. Our CRM uses fixed L_2 norm clipping with supervised loss, requiring no RL or online rollouts. Conformal prediction [25] has been applied to robotics for

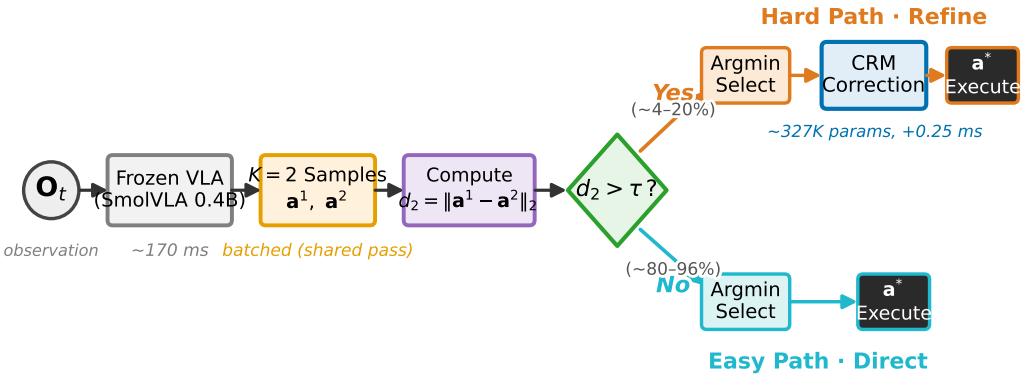


Figure 1: Adaptive sample-select-correct pipeline. K=2 action samples are drawn from the frozen VLA; the argmin-to-centroid sample is selected, and a lightweight CRM applies bounded correction ($\|\boldsymbol{\delta}\|_2 \leq \varepsilon$) only when the pairwise L_2 distance exceeds the uncertainty threshold τ.

uncertainty-aware control; our adaptive threshold τ draws on this calibration philosophy, though the CRM itself conditions only on observation features.

Inference-time compute scaling. Test-time scaling in LLMs [4] improves quality via best-of-N selection. RoVer [26] transfers this to VLAs with a reward model; DA-SIP [27] adapts denoising steps per difficulty. Our analysis shows a different pattern in continuous action spaces: adaptive gating (knowing *when* to correct) is the key mechanism, as unconditional correction is unreliable across suites while selection alone is insufficient (Section 4.2). This motivates a K=2 uncertainty probe that triggers correction rather than scaling sample count.

3 ACR-VLA Pipeline

ACR-VLA adaptively allocates inference-time compute for a frozen flow-matching VLA (Figure 1). At each control step, a K=2 uncertainty probe determines whether the VLA is confident or uncertain; confident steps output the argmin-to-centroid sample directly, while uncertain steps apply a learned residual correction via a lightweight Correction Residual Module (CRM). The base VLA is never retrained.

3.1 Problem Setup

Let π_θ be a frozen flow-matching VLA that, given an observation o_t (image, language instruction, proprioceptive state), generates action chunks of length C=50 in $\mathbb{R}^{d \times C}$ with d=6 (3 translation + 3 rotation). Under receding-horizon execution, only the first H=5 steps of each chunk influence the robot before re-planning, so we discard the remainder. The gripper dimension is handled separately via direct thresholding and is excluded from the pipeline.

Each forward pass of π_θ produces a stochastic action sample; querying independently K times yields K candidate chunks. Our goal is to produce a single action more accurate than any individual sample, with no changes to π_θ. The method applies to any flow-matching VLA that produces diverse samples (Appendix A).

3.2 Multi-Sample Selection

We draw K action chunks from π_θ via independent flow-matching rollouts and truncate each to H=5 steps, yielding samples $\{a_h^{(k)}\}_{k=1}^K$ with $a_h^{(k)} \in \mathbb{R}^d$ for horizon step $h = 1, \ldots, H$. For each step, we compute the sample mean $\boldsymbol{\mu}_h = \frac{1}{K} \sum_{k=1}^K a_h^{(k)}$ and select the sample with the smallest total

L_2 distance to the mean across all horizon steps:

$$a_{\text{base}} = \underset{k \in \{1,\dots,K\}}{\arg\min} \sum_{h=1}^{H} \left\| a_h^{(k)} - \boldsymbol{\mu}_h \right\|_2^2. \tag{1}$$

Unlike the element-wise mean, which can blend incompatible modes, the argmin returns an actual sample and preserves within-sample temporal coherence.

3.3 Correction Module (CRM)

The CRM is a lightweight MLP that refines the selected action a_{base} by predicting a bounded residual correction conditioned on the current observation and the selected action itself.

Architecture. The CRM consists of three fully connected layers with ReLU activations: $\text{Linear}(n_{\text{in}}, 256) \rightarrow \text{ReLU} \rightarrow \text{Linear}(256, 256) \rightarrow \text{ReLU} \rightarrow \text{Linear}(256, H{\cdot}d)$, preceded by input normalization. The input is the concatenation of the frozen observation embedding $\mathbf{z}_{\text{obs}} \in \mathbb{R}^{n_{\text{obs}}}$ from the VLA's vision-language encoder and the selected action $a_{\text{base}} \in \mathbb{R}^{H \cdot d}$ (H=5 steps $\times$ d=6 DOFs), giving $n_{\text{in}} = n_{\text{obs}} + 30$. The architecture is identical across VLAs; only n_{obs} differs (960 for SmolVLA, 2048 for $\pi_{0.5}$), yielding 327K–615K parameters ($< 1\%$ of either backbone). The output is a residual $\boldsymbol{\delta} \in \mathbb{R}^{30}$, and the corrected action is:

$$\mathbf{a}^* = a_{\text{base}} + \text{clip}\big(\boldsymbol{\delta},\ \|\boldsymbol{\delta}\|_2 \le \varepsilon\big), \tag{2}$$

where ε is a norm bound that limits the correction magnitude (Section 3.4). The CRM never sees individual action candidates: it receives only the already-selected a_{base} and predicts a correction toward the expert action. We tested alternative conditioning inputs (per-DOF conformal features, scalar uncertainty); observation-only conditioning performs best and is the primary variant.

Training. The CRM is trained offline on demonstration trajectories from LIBERO [9] with L_2 loss:

$$\mathcal{L} = \left\| \mathbf{a}^* - \mathbf{a}_{\text{expert}} \right\|_2^2, \tag{3}$$

where $\mathbf{a}_{\text{expert}}$ is the demonstration action. For each VLA, we precompute training data from 10 demonstrations per task across the target LIBERO suites. At each demonstration step, we run K=10 forward passes through the frozen VLA, apply argmin selection (1) to obtain a_{base}, and train the CRM to predict the residual toward $\mathbf{a}_{\text{expert}}$. Training uses Adam (lr=10^{-3}), batch size 32, 50 epochs, with Gaussian noise augmentation on a_{base} to improve generalization to the closed-loop state distribution [28].

3.4 Magnitude Control

The norm bound ε in (2) is critical for stable closed-loop performance. Without clipping, the CRM's unconstrained output norm ($\approx$1.42) causes compounding errors. Constraining ε to $[0.05, 0.2]$ yields stable corrections. The adaptive pipeline uses ε=0.2 (correction gated to high-uncertainty steps only), while unconditional correction requires a tighter ε=0.05 to limit overcorrection (Section 4.3).

3.5 Adaptive Inference-Time Compute

Experiments reveal that unconditional correction is unstable: CRM applied at every step harms some suites (-1.2 pp) while helping others, even with a conservative norm bound (ε=0.05; Section 4.3). The $\sim$80–90% of steps where the VLA is already confident receive corrections that are noise. Fixed K=10 multi-sample selection offers no remedy; selection without learned correction yields at most $+0.5$ pp and hurts on some suites. The solution is a per-step uncertainty probe that decides *when* to apply correction.

Why K=2. K=2 is the minimum sample size that admits a pairwise disagreement measure, and under batched inference the uncertainty signal is *free*: K=2 latency ($\sim$170 ms) matches K=1 because both samples share the same prefix encoding and flow-matching denoising runs in parallel (Table 2). Empirically, the K=2 L_2 distance achieves ρ=0.9979 with K=10 sample variance; three alternative probes, including CRM correction magnitude (K=1 and K=2) and flow-matching velocity dispersion (FMVD), all fail ($\rho < 0.16$; Appendix C). Despite using 5$\times$ fewer samples, K=2 with adaptive gating matches or exceeds K=10 unconditional correction on all evaluated suites (Section 4.2).

Three-step adaptive pipeline. At each control step, the pipeline executes:

1. **Uncertainty probe.** Draw K=2 samples $a^{(1)}, a^{(2)}$ from π_θ in a single batched forward pass. Compute the centroid $\mathbf{c} = (a^{(1)} + a^{(2)})/2$ and the pairwise uncertainty $d_2 = \|a^{(1)} - a^{(2)}\|_2$. Select $\hat{a} = \arg\min_{a \in \{a^{(1)}, a^{(2)}\}} \|a - \mathbf{c}\|_2$.
2. **Adaptive decision.** If $d_2 \leq \tau$ (low uncertainty), output $\hat{a}$ directly, requiring no further computation. If $d_2 > \tau$ (high uncertainty), pass $\hat{a}$ to the CRM for correction.
3. **Learned correction.** For high-uncertainty steps only, the CRM predicts a bounded residual: $\hat{a}_{\text{out}} = \hat{a} + \boldsymbol{\delta}$, where $\boldsymbol{\delta} = \text{CRM}(o_t, \hat{a})$ and $\|\boldsymbol{\delta}\|_2 \leq \varepsilon$ (2). The pipeline uses at most K=2 forward passes; performance gains come from learned correction, not brute-force sampling.

The threshold τ is a percentile of the empirical d_2 distribution from CRM training data. Higher τ triggers correction on fewer steps; the optimal percentile is task-difficulty-dependent (Section 4.4). CRM inference adds $\sim$0.25 ms; the VLA forward pass ($\sim$170 ms batched) dominates latency. We use the term "conformal" loosely to denote calibrated thresholding on held-out data; ACR-VLA does not implement formal conformal prediction guarantees [25].

4 Experiments

We evaluate ACR-VLA on SmolVLA [2] (0.4B) across four LIBERO [9] task suites, comparing five methods that isolate the contributions of selection, correction, and adaptive gating. We ask: (i) does adaptive gating produce positive improvements where unconditional correction and selection-only approaches fail? (ii) what is the computational cost of the uncertainty probe?

4.1 Setup

Benchmarks. We use all four LIBERO suites: Spatial (10 tasks, spatial reasoning), Object (10 tasks, novel object instances), Goal (10 tasks, goal specification), and Long (10 tasks, long-horizon manipulation). Each task is evaluated over 20 rollouts (max 300 steps, 200 per suite); success is binary.

Base VLA. SmolVLA [2] (0.4B parameters, flow matching) uses dual-camera RGB (256$\times$256), 8D proprioceptive state, and produces 7D actions (6D pose + 1D gripper, thresholded separately). We additionally report $\pi_{0.5}$ [10] (3.6B) baselines and OpenVLA-OFT [11] (7B) boundary analysis in Section 4.5.

Methods. All methods use the same frozen SmolVLA; no VLA weights are modified:

- **VLA-only** (K=1): single forward pass, no correction.
- **CRM always-on** (K=10, ε=0.05): argmin selection + unconditional CRM correction at every step. Uses a conservative norm bound (ε=0.05) to limit overcorrection.
- K=10 **argmin** [6] (K=10): argmin-to-centroid selection over 10 samples, no learned correction. Represents the selection-only approach.
- **ACR-VLA** (K=2, ε=0.2, τ=0.25): the proposed adaptive pipeline (Section 3.5). CRM correction applied only when $d_2 > \tau$.
- K=2 **mean**: centroid of two samples, no CRM. Isolates the multi-sampling effect.

Table 1: Task success rate (%↑) on SmolVLA across four LIBERO suites. Paired differences (Adaptive−VLA) are directionally consistent but not statistically significant at α=0.05 (n=3 matched seeds per suite). Δ_{CRM}: Adaptive−CRM always-on. Trigger: percentage of steps where the adaptive gate fires.

Method	K	Spatial (%)	Object (%)	Goal (%)	Long (%)
VLA-only	1	$79.3 \pm 0.8^{(3)}$	$77.7 \pm 1.3^{(3)}$	$83.7 \pm 1.3^{(3)}$	$58.0 \pm 1.5^{(3)}$
CRM always-on	10	$79.8 \pm 1.3^{(3)}$	$76.5 \pm 2.3^{(3)}$	$83.7 \pm 1.8^{(3)}$	$55.8 \pm 1.2^{(3)}$
K=10 argmin	10	$80.3 \pm 1.0^{(3)}$	$77.3 \pm 1.5^{(3)}$	$80.8 \pm 0.3^{(3)}$	$56.5^{(1)}$
K=2 mean	2	—	$77.5 \pm 2.3^{(3)}$	$83.0 \pm 2.2^{(3)}$	$57.0^{(1)}$
ACR-VLA	2	$80.8 \pm 1.1^{(3)}$	$77.8 \pm 2.1^{(5)}$	$85.0 \pm 1.3^{(3)}$	$57.7 \pm 1.5^{(3)}$
Δ vs VLA-only		+1.5	+0.1	+1.3	−0.3
Δ_{CRM} vs CRM		+1.0	+1.3	+1.3	+1.9

— = omitted (computational budget); $^{(n)}$ = number of seeds.

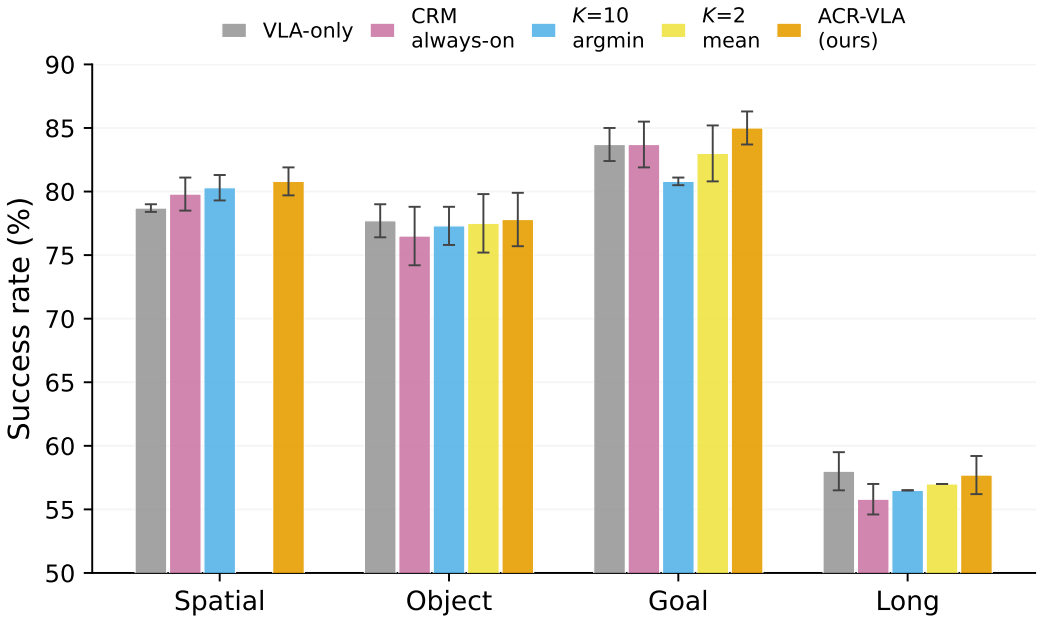


Figure 2: Success rates across five methods and four LIBERO suites. ACR-VLA (orange) matches or exceeds all baselines on every suite. CRM always-on (pink) degrades on Object and Long; K=10 argmin (blue) degrades on Goal. Error bars: ±1 std over available seeds.

CRM training. The CRM (∼327K parameters) is trained on LIBERO demonstrations with 10 demos per task, K=10 forward passes each (Adam lr=10^{-3}, batch 32, 50 epochs, ε=0.2 for adaptive / ε=0.05 for always-on). For Spatial, Object, and Goal, the CRM is trained on three suites (30 tasks, ∼168K samples); for Long, we use a CRM trained on all four suites (40 tasks, ∼224K samples) to include Long-horizon demonstrations in the training distribution (Appendix D).

Statistics. We report mean ± std over n seeds. Paired improvements (Adaptive vs. VLA-only) are not statistically significant at conventional thresholds (n=3 matched seeds per suite). We defer to Section 4.3 for within-suite controlled analysis of the gating mechanism via τ-sweeps.

4.2 Main Results

Table 1 and Figure 2 report task success rates across all methods and suites. ACR-VLA's adaptive gate correctly modulates intervention intensity: on suites where the VLA shows high uncertainty

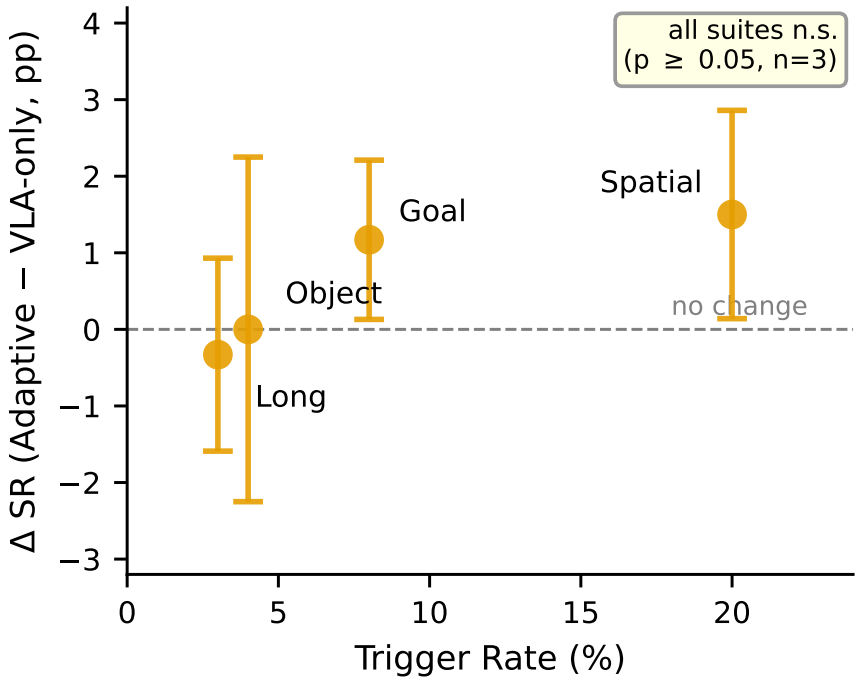


Figure 3: Trigger rate vs. improvement over VLA-only. Suites where the gate fires on >7% of steps (Spatial, Goal) see improvement; suites where the VLA is already confident (Object, Long, trigger <5%) preserve baseline. The gate correctly partitions correction budget by per-step uncertainty.

(trigger rate >7%), the gate fires and the CRM applies corrections (+1.5 pp on Spatial, +1.3 pp on Goal, 3 seeds each). On suites where the VLA is already confident (trigger rate <5%), the gate rarely fires and the output matches baseline: Object (Δ=+0.1 pp, 5 seeds) and Long (Δ=−0.3 pp, 3 seeds). This avoids the degradation caused by unconditional correction (−1.2 pp Object, −2.2 pp Long).

Unconditional correction is unstable across suites. *CRM always-on* (ε=0.05) applies correction at every step: −2.2 pp on Long (55.8 ± 1.2%, 3 seeds), −1.2 pp on Object (76.5 ± 2.3%, 3 seeds), $\Delta\approx$0 on Goal (83.7 ± 1.8%, 3 seeds), and +1.1 pp on Spatial. Without gating, the CRM hurts on two suites while providing inconsistent benefit elsewhere. K=10 *argmin* invests 10× the VLA forward passes but hurts on Goal (−2.9 pp, 80.8 ± 0.3%, 3 seeds). Selection without correction is insufficient. K=2 *mean* offers no learned correction mechanism (−0.7 pp Goal, −1.0 pp Long; Long is single-seed preliminary), isolating that multi-sampling alone is not sufficient.

Statistical note. Paired differences (Adaptive−VLA) are directionally consistent (positive on 3 of 4 suites) but do not reach $p < 0.05$ with n=3 matched seeds (Spatial: $p\approx$0.27). The stronger claim is Adaptive vs. CRM always-on: ACR-VLA exceeds unconditional correction on all four suites (Δ_{CRM}=+1.0 to +1.9 pp), though individual-suite differences remain within measurement uncertainty (n=3). This confirms that gating prevents the degradation caused by unconditional correction. An extended 8-seed analysis on Spatial yields p=0.017 (Appendix E).

4.3 Why Gating Is Necessary

Figure 3 visualizes the mechanism: the gate's trigger rate correlates monotonically with improvement. Suites where the VLA shows high uncertainty (trigger rate >7%: Spatial, Goal) see positive corrections; suites where the VLA is already confident (trigger rate <5%: Object, Long) preserve baseline. The gate correctly partitions correction budget by per-step uncertainty, avoiding the

Table 2: Computational comparison. ACR-VLA uses $5\times$ fewer VLA forward passes than $K{=}10$ methods while achieving higher success rates. CRM overhead is negligible.

Method	VLA calls	Latency (ms)	CRM calls (%)
VLA-only	1	$\sim$170	0
ACR-VLA	**2**	$\sim$**170**	**4–20**
$K{=}10$ argmin	10	$\sim$170 (batched)	0
CRM always-on	10	$\sim$170 (batched)	100

Table 3: Cross-VLA analysis: $\pi_{0.5}$ (3.6B) on LIBERO ($n{=}3$ seeds). Near-expert baselines (94–99%) are degraded by unconditional CRM correction; adaptive gating recovers 19–38% of the loss on 3/4 suites.

Suite	VLA-only ($K{=}1$)	CRM ($K{=}2$)	ACR-VLA ($K{=}2$)
Spatial	98.0 ± 0.5	96.8 ± 0.3	97.2 ± 0.8
Goal	96.7 ± 1.0	83.5 ± 1.3	88.5 ± 1.7
Long	95.0 ± 0.9	78.0 ± 3.5	81.3 ± 1.4
Object	99.7 ± 0.3	88.0 ± 1.0	86.2 ± 1.6

-1.2 pp (Object) and -2.2 pp (Long) degradation of unconditional correction. Threshold sweeps on two suites confirm an inverted-U response (Appendix B): too much correction is as harmful as too little. The $K{=}10$ argmin baseline hurts on Goal (-2.9 pp) and Long (-1.5 pp): selection alone cannot fix systematic errors. A $K{=}1$ ablation applies CRM unconditionally (no pairwise uncertainty available for gating), yielding SR=79.5% vs. $K{=}2$ adaptive at 80.8% ($\Delta{=}{+}1.3$ pp); the gap reflects the combined benefit of multi-sample centering and selective gating.

4.4 Efficiency

Table 2 compares computational cost. Under batched inference with shared prefix encoding, $K{=}2$ latency matches $K{=}1$ ($\sim$170 ms), making the uncertainty probe computationally free. CRM inference adds $\sim$0.25 ms ($< 0.2\%$ overhead). ACR-VLA triggers CRM on 4–20% of steps (4% on Object to 20% on Spatial), using $5\times$ fewer VLA forward passes than $K{=}10$ methods while achieving higher success rates. The efficiency advantage is structural: the adaptive gate allocates correction budget to the steps that need it.

4.5 Cross-VLA Analysis: Model Capability Dependency

We evaluate ACR-VLA on $\pi_{0.5}$ [10] (3.6B, flow matching), where baselines reach 94–99% (Table 3). Note that $\pi_{0.5}$ uses $K{=}2$ CRM (matching ACR-VLA's sample budget) rather than $K{=}10$ as in Table 1, to isolate the effect of gating from sample count. At this capability level, CRM correction ($K{=}2$, $\varepsilon{=}0.2$) degrades all four suites (-1.2 to -17 pp); the correction norm saturates at its cap on every suite. Adaptive gating recovers on 3/4 suites (33–38% on Spatial/Goal, 19% on Long; the lower rate is consistent with error compounding over longer horizons). Object is the exception: at 99.7% baseline, $K{=}2$ selection noise outweighs recovery. This reveals a model capability dependency: adaptive gating limits damage for near-expert VLAs, but correction provides diminishing returns as baseline capability increases. A boundary analysis on OpenVLA-OFT [11] (7B, deterministic) confirms that stochastic diversity is a prerequisite (Appendix A).

5 Conclusion

We presented ACR-VLA, an adaptive inference-time correction pipeline for frozen flow-matching VLAs. A $K{=}2$ pairwise L_2 uncertainty probe ($\rho{=}0.9979$) gates a lightweight CRM ($\sim$327K parameters). On SmolVLA (0.4B), unconditional correction degrades 2/4 suites (-2.2 pp Long, -1.2 pp Object); on $\pi_{0.5}$ (3.6B, 94–99% baseline), it degrades all four (up to -17 pp). ACR-VLA's adaptive gate recovers performance in both settings: exceeding unconditional $K{=}10$ correction on all SmolVLA suites with $5\times$ fewer forward passes, and recovering 19–38% of the CRM-induced loss on 3/4 $\pi_{0.5}$ suites. The key finding is mechanistic: adaptive gating modulates intervention intensity, turning an unstable correction module into a reliable one.

6 Limitations

Effect sizes and statistical power. Paired improvements (Adaptive vs. VLA-only) are directionally consistent across suites but do not reach statistical significance at α=0.05 with n=3 matched seeds (Spatial: $p\approx$0.27). Per-suite sample sizes are limited by computational budget; the consistent directional pattern across 4 suites provides corroborating but not definitive evidence of improvement. The stronger empirical claim is Adaptive vs. CRM always-on, where ACR-VLA exceeds unconditional correction on all four suites (Δ_{CRM}=+1.0 to +1.9 pp).

Confidently wrong steps. The pairwise L_2 probe measures sample disagreement, not correctness. On Long (trigger rate $\sim$4%), the VLA may produce confident but systematically incorrect outputs that escape gating.

Scope and generalization. The CRM requires task-specific calibration data (10 demos per task); deterministic VLAs lie outside scope (Appendix A). All experiments use LIBERO simulation; real-robot transfer remains future work. The $\pi_{0.5}$ analysis (Section 4.5) reveals diminishing returns as baseline capability increases; broader cross-architecture validation is needed.

AI tools assisted with code and manuscript.

References

[1] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky. π_0: A vision-language-action flow model for general robot control, 2024.

[2] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene. Smolvla: A vision-language-action model for affordable and efficient robotics, 2025.

[3] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.

[4] C. Snell, J. Lee, K. Xu, and A. Kumar. Scaling llm test-time compute optimally can be more effective than scaling parameters for reasoning. In *International Conference on Learning Representations (ICLR)*, 2025.

[5] K. Sendai, M. Alvarez, T. Matsushima, Y. Matsuo, and Y. Iwasawa. Leave no observation behind: Real-time correction for vla action chunks, 2025.

[6] L. Chen, Z. Lyu, and W. J. Beksi. Reconvla: An uncertainty-guided and failure-aware vision-language-action framework for robotic control, 2026.

[7] C. Liu, W. Tan, L. Zhu, F. Li, J. Li, G. Yang, and H. T. Shen. Self-correcting vla: Online action refinement via sparse world imagination, 2026.

[8] Z. Li, J. Liu, Z. Dong, T. Teng, Q. Rouxel, D. Caldwell, and F. Chen. Towards deploying vla without fine-tuning: Plug-and-play inference-time vla policy steering via embodied evolutionary diffusion, 2025.

[9] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[10] P. Intelligence, K. Black, N. Brown, J. Bradbury, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, O. Mees, M. Mothukuri, S. Nair, K. Pertsch, A. Ren, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, and H. Wang. $\pi_{0.5}$: a vision-language-action model with open-world generalization, 2025.

[11] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. Openvla: An open-source vision-language-action model, 2024.

[12] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, P. Florence, C. Fu, M. G. Arenas, K. Gopalakrishnan, K. Han, K. Hausman, A. Herzog, J. Hsu, B. Ichter, A. Irpan, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, L. Lee, T.-W. E. Lee, S. Levine, Y. Lu, H. Michalewski, I. Mordatch, K. Pertsch, K. Rao, K. Reymann, M. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wohlhart, J. Wu, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich. Rt-2: Vision-language-action models transfer web knowledge to robotic control, 2023.

[13] D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, J. Luo, Y. L. Tan, L. Y. Chen, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, and S. Levine. Octo: An open-source generalist robot policy, 2024.

[14] C. Chi, Z. Xu, S. Feng, E. Cousineau, B. Burchfiel, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems (RSS)*, 2023. Also published in IJRR 2024. arXiv:2303.04137.

[15] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei, J. Fu, J. Gong, and X. Qiu. Libero-plus: In-depth robustness analysis of vision-language-action models, 2025.

[16] S. Fei, S. Wang, L. Ji, A. Li, S. Zhang, L. Liu, J. Hou, J. Gong, X. Zhao, and X. Qiu. Srpo: Self-referential policy optimization for vision-language-action models, 2025.

[17] H. Zhang, S. Zhang, J. Jin, Q. Zeng, R. Li, and D. Wang. Robustvla: Robustness-aware reinforcement post-training for vision-language-action models, 2025.

[18] W. Zhang, Y. Wang, H. Luo, H. Yuan, Y. Feng, S. Zheng, Q. Jin, and Z. Lu. Dig-flow: Discrepancy-guided flow matching for robust vla models, 2025.

[19] C. Liu, Y. Liu, T. Wang, Q. Zhuang, J. C. Liang, W. Yang, R. Xu, Q. Wang, D. Liu, and C. Han. On-the-fly vla adaptation via test-time reinforcement learning, 2026.

[20] T. Silver, K. Allen, J. Tenenbaum, and L. Kaelbling. Residual policy learning. In *Robotics: Science and Systems (RSS)*, 2018. arXiv:1812.06298.

[21] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine. Residual reinforcement learning for robot control. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 6023–6029, 2019.

[22] Y. Jiang, C. Wang, R. Zhang, J. Wu, and L. Fei-Fei. Transic: Sim-to-real policy transfer by learning from online correction. In *Conference on Robot Learning (CoRL)*, 2024. arXiv:2405.10315.

[23] W. Xiao, H. Lin, A. Peng, H. Xue, T. He, Y. Xie, F. Hu, J. Wu, Z. Luo, L. Fan, G. Shi, and Y. Zhu. Self-improving vision-language-action models with data generation via residual rl. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2511.00091.

[24] C. Pan, H. H. Cheng, and J. Hughes. Online imitation learning for manipulation via decaying relative correction through teleoperation, 2025.

[25] A. N. Angelopoulos and S. Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *Foundations and Trends in Machine Learning*, 2023.

[26] Y. Dai, H. Liu, Z. Bai, Y. Liu, J. Wang, H. Su, C. Chen, J. Lin, and L. Wu. Rover: Robot reward model as test-time verifier for vision-language-action model, 2025.

[27] S. Chun, J. Lee, M. S. Albergo, L. Xie, and E. Vanden-Eijnden. Dynamic test-time compute scaling in control policy: Difficulty-aware stochastic interpolant policy. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[28] S. Ross, G. J. Gordon, and J. A. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.

A Applicability Boundary: Stochastic VLA Diversity

The multi-sample pipeline requires stochastic action diversity across samples. We validate this prerequisite on two VLA architectures: SmolVLA [2] (flow matching, 0.4B) and OpenVLA-OFT [11] (deterministic L_1 regression, 7B).

SmolVLA produces meaningfully diverse action samples across all dimensions via its flow-matching generative process. In contrast, OpenVLA-OFT's deterministic L_1 regression produces near-identical outputs: mean pairwise L_2=0.00436 across K=2 samples, with 97.3% of steps below 0.01. The uncertainty gate never fires, and multi-sample selection is vacuous.

This result establishes an architectural boundary: multi-sample inference-time correction is specific to stochastic generative VLAs (flow matching, diffusion). Deterministic VLAs lack the action diversity that selection and correction rely on. The boundary is architectural, not a limitation of our method: any inference-time approach that draws multiple action candidates requires the base policy to produce diverse samples.

B Threshold Sensitivity Analysis

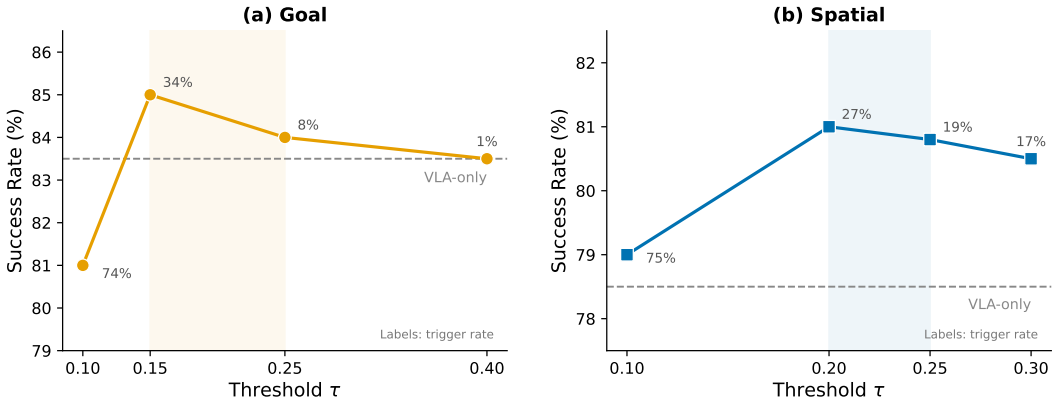


Figure 4: Threshold τ controls trigger rate, which controls improvement: inverted-U on both Goal and Spatial (seed 42, ε=0.2). Low τ overcorrects (>70% trigger); high τ disables correction. The optimal region $\tau \in [0.20, 0.25]$ is consistent across suites. Labels show trigger rate at each τ.

Table 4: Threshold sensitivity on Goal and Spatial suites (seed 42, ε=0.2). Seed-matched VLA-only baselines: Goal 83.5%, Spatial 78.5%. Both suites exhibit an inverted-U pattern with consistent optimal region $\tau \in [0.20, 0.25]$.

	Goal		Spatial	
τ	Trigger (%)	SR (%)	Trigger (%)	SR (%)
0.10	73.9	81.0 (−2.5)	75.2	79.0 (+0.5)
0.15	33.9	85.0 (+1.5)	—	—
0.20	—	—	26.6	81.0 (+2.5)
0.25	7–9	84.0 (+0.5)	∼19	80.8 (+2.3)
0.30	—	—	16.7	80.5 (+2.0)
0.40	1.2	83.5 (0.0)	—	—

Table 4 reports success rates on Goal and Spatial (seed 42) as a function of the uncertainty threshold τ, with ε=0.2 (the adaptive correction magnitude). Both suites exhibit an inverted-U pattern. On Goal, τ=0.10 triggers on 73.9% of steps and degrades by −2.5 pp, while τ=0.40 triggers on 1.2% and recovers VLA-only performance. On Spatial, τ=0.10 triggers on 75.2% but yields only +0.5 pp; the peak is at τ=0.20 (+2.5 pp, 26.6% trigger). The consistent optimal region

$\tau \in [0.20, 0.25]$ across both suites supports the mechanism claim: τ controls trigger rate, which controls improvement.

The main experiments use τ=0.25, which balances correction coverage and selectivity across suites. The inverted-U shape confirms that gating selectivity, not correction strength, drives performance: too much correction is as harmful as too little.

Note on ε comparisons. The τ-sweep uses ε=0.2 (the adaptive pipeline's norm bound). CRM always-on in Table 1 uses a tighter ε=0.05 to limit overcorrection when applied unconditionally. The two settings are not directly comparable: CRM always-on (ε=0.05, 100% trigger) achieves 85.5% on Goal seed 42, exceeding τ=0.10 (ε=0.2, 73.9% trigger) at 81.0%. This reflects the interaction between correction magnitude and trigger frequency: unconditional correction requires a gentler ε to remain stable, while gated correction tolerates a larger ε because it fires selectively.

C Uncertainty Proxy Comparison

Table 5: Spearman ρ between candidate uncertainty proxies and K=10 action variance (ground-truth uncertainty). Evaluated on LIBERO-Object (10 tasks $\times$ 10 rollouts, 27,626 steps). Only pairwise L_2 distance achieves meaningful correlation.

Proxy	Spearman ρ	Extra VLA calls
K=2 pairwise L_2 (ours)	**0.9979**	1
K=1 CRM correction magnitude	0.152	0
K=2 CRM correction magnitude	0.134	1
Flow velocity dispersion (FMVD)	0.083	0

Table 5 compares four candidate uncertainty proxies against K=10 sample variance as ground truth. CRM correction magnitude (the L_2 norm of the CRM's predicted residual) is attractive because it requires no extra VLA forward passes, but correlates weakly with true uncertainty ($\rho < 0.16$): the CRM predicts similar-magnitude corrections regardless of whether the VLA is confident or uncertain. FMVD, which measures dispersion in the flow-matching velocity field, performs worse still (ρ=0.083). The K=2 pairwise L_2 distance achieves near-perfect correlation (ρ=0.9979) at the cost of one additional VLA forward pass, which is absorbed by batched inference (Section 4.4).

D Reproducibility Details

Hardware. All experiments run on NVIDIA RTX 4090 GPUs (24 GB VRAM). SmolVLA inference uses a single GPU; CRM training and evaluation each use a single GPU.

CRM training. The 3-suite CRM (Spatial, Object, Goal; $\sim$168K samples) trains for 50 epochs with Adam (lr=10^{-3}, batch size 32) in $\sim$6 GPU-hours on 1$\times$ RTX 4090. The 4-suite CRM (adding Long; $\sim$224K samples) trains in $\sim$8 GPU-hours. Precomputing the K=10 forward-pass cache (required once per suite) takes $\sim$2 GPU-hours per suite.

Evaluation. Each suite evaluation (20 rollouts $\times$ 10 tasks $\times$ 300 max steps) takes $\sim$2.5 GPU-hours. A full 5-method $\times$ 4-suite $\times$ 3-seed experiment requires $\sim$150 GPU-hours total.

Software. PyTorch 2.4, Transformers 4.45, LIBERO (MuJoCo-based). Code and CRM checkpoints will be released upon acceptance.

E Extended 8-Seed Analysis on Spatial

Table 6 reports per-seed results for the extended 8-seed paired analysis on LIBERO-Spatial. A paired t-test yields Δ=+1.4 pp, p=0.017 (n=8), reaching statistical significance at α=0.05. Six

Table 6: Per-seed success rates (%) on LIBERO-Spatial for VLA-only vs. ACR-VLA (K=2, τ=0.25). All 8 seeds use identical K=2 configuration (code-verified).

Seed	VLA-only	ACR-VLA	Δ
42	80.0	82.0	$+2.0$
43	79.0	80.0	$+1.0$
44	78.5	80.5	$+2.0$
45	80.0	80.0	$+0.0$
46	80.5	80.0	-0.5
47	80.0	81.5	$+1.5$
48	79.5	81.0	$+1.5$
49	78.5	82.0	$+3.5$
Mean±std	79.5 ± 0.8	80.9 ± 0.9	$+1.4$

of eight seeds show positive Δ; the two non-positive seeds (Δ=0.0 and Δ=-0.5) are consistent with measurement noise rather than systematic degradation. This analysis provides the strongest statistical evidence that adaptive gating produces a genuine improvement on the suite where the uncertainty gate fires most frequently (trigger rate $\sim$20%).